Zero-overhead abstractions



Why do many programming languages have `null`?

The usability of `null`

```cpp
#include <iostream>

void foo(int* ptr) {
    std::cout << *ptr << std::endl;
}
int main() {
    int bar = 42;
    foo(&bar);
    foo(nullptr);
    return 0;
}
```

What could be improved?

Problems of `null`

- `null` can mean multiple things: Not initialized, does not exist, not needed
- For pointers, two things mix: Indirection with absence of a value
- On the hardware level, `null` is a sentinel value (typically `0x0`)
- On some platforms, `0x0` might be a valid memory address!

Fixing it - v1?

```
1  struct MaybePtr {
2      MaybePtr() : _exists(false) {}
3      explicit MaybePtr(int* ptr): _ptr(ptr), _exists(true) {}
4
5      int* get() const {
6          if(!_exists) {
7              throw std::expection("Value does not exist!");
8          }
9          return _ptr;
10     }
11 private:
12     int* _ptr;
13     bool _exists;
14 }
```

- Not ideal:
- Runtime error instead of compile time check
- Some memory overhead

Zero-overhead abstraction

- We want something like `null` with better ergonomics (i.e. an abstraction) but as efficient as a hand-written solution (i.e. `if(!ptr)`)
- Such abstractions (if they even exist) are called zero-overhead abstractions [Stroustrup, 2000]
- Functions and local variables are zero-overhead abstractions!



Speaker notes

